TaskFlow Backend Proposal Documentation

Executive Summary

TaskFlow is a comprehensive task management platform designed specifically for student development teams. Its backend provides the foundation required to manage projects, tasks, users, files, communication, notifications, and performance information in one coordinated system.

The proposed backend is designed around a modern layered architecture that separates the client-facing API, application logic, data access, and database services. This structure makes the system easier to maintain and extend while supporting secure authentication, role-based access, real-time collaboration, efficient data handling, and reliable storage.

The implementation will use Next.js and TypeScript together with MySQL, Redis, JWT authentication, cloud file storage, and WebSocket-based real-time communication. The proposal also defines the development roadmap, security measures, performance targets, testing requirements, maintenance approach, and key risks so that the platform can be developed in a controlled and scalable manner.

1. Project Overview

1.1 Purpose

The TaskFlow backend serves as the foundation of a collaborative task management platform. It enables student development teams to organize projects and tasks efficiently, collaborate in real time, monitor progress and performance, share files, and communicate within the platform. By centralizing these activities, the backend provides the services required for the frontend application to operate consistently and securely.

1.2 Target Audience

TaskFlow is intended primarily for university students working on software development projects, student development teams and clubs, academic institutions running software engineering programs, and small development teams that need a lightweight project management solution.

1.3 Core Objectives

The core objectives are to provide a secure and reliable API for frontend applications, enable real-time collaboration and updates, maintain data integrity and consistency, support multiple teams and projects as the system grows, and maintain high performance and availability.

2. System Architecture

2.1 Architecture Overview

The backend follows a modern layered architecture with a clear separation of responsibilities. Frontend applications communicate with the API Gateway through HTTPS or WebSockets. The API Gateway manages authentication, rate limiting, and request routing before requests are passed to the Application Layer, where business rules, validation, and services are handled. The Data Access Layer manages database interaction

through an ORM, query building, and data validation, while the Database Layer provides MySQL for primary data storage and Redis for caching and session-related operations.

2.2 Key Components

API Gateway. The API Gateway is the main entry point for client requests. It routes requests to the appropriate backend services while supporting authentication and rate limiting.

Authentication Service. This service manages user authentication, JWT tokens, session handling, password security, and related account access processes.

Task Service. The Task Service provides the core operations for creating, viewing, updating, and deleting tasks and managing their related information.

Project Service. The Project Service manages project creation, project information, team assignments, project status, and other project-level operations.

User Service. The User Service manages user profiles, roles, and team membership information.

Notification Service. The Notification Service manages real-time notifications, alerts, and other messages that need to reach users.

File Service. The File Service handles file uploads, storage, organization, retrieval, and access permissions.

Reporting Service. The Reporting Service produces analytics and performance reports that help teams monitor their projects and activities.

Cache Service. The Cache Service uses Redis to improve performance by storing frequently accessed information and session data.

3. Core Features

3.1 User Management

User management provides registration and authentication, role-based access control, profile management, avatar uploads, password reset, email verification, and session management. The system supports four main roles. Administrators have full system access and can manage users and system configuration. Project Managers can create projects, assign tasks, and manage team members. Developers can create and update tasks, comment, and upload files. Viewers have read-only access to assigned projects.

3.2 Task Management

Task management supports the complete task lifecycle, including creating, reading, updating, and deleting tasks. Users can track tasks through statuses such as To Do, In Progress, Review, and Done, while priority levels can be set to Low, Medium, High, or Urgent. The system also supports task assignment and reassignment, due dates, task dependencies, comments and discussions, file attachments, task history and audit logs, subtasks, and reusable task templates.

3.3 Project Management

Project management provides the structure needed to organize development work and monitor project progress. It supports project creation and configuration, team member assignment, visual progress tracking, project status management, sprint or iteration planning, timelines, milestones, analytics, reports, and project templates.

3.4 Team Collaboration

Team collaboration features are designed to keep members informed and connected while work is being completed. The platform supports:

- Team member management
- Real-time activity feeds
- In-app messaging and notifications
- Comment threads on tasks
- File sharing within teams
- Team performance analytics
- A team calendar

3.5 File Management

File management allows teams to store, organize, share, and retrieve project files from within the platform. It supports:

- File upload and download
- File storage and organization
- File version control
- Preview of images, PDFs, and documents
- File sharing and permissions
- Storage quota management
- File search

3.6 Notifications

The notification system keeps users updated about relevant activities and changes. It supports:

- Real-time notifications
- Email notifications
- Notification preferences
- An in-app notification center
- Push notifications for mobile devices
- Smart notifications based on user roles

4. Technical Specifications

4.1 Technology Stack

Framework: Next.js 16 (App Router). Provides backend API routes and server-side rendering.

Language: TypeScript. Provides type safety and a better developer experience.

Database: MySQL 8.0. Provides primary data storage.

Cache: Redis. Supports session storage, caching, and real-time data requirements.

Authentication: JWT. Provides stateless authentication.

File Storage: Cloudinary / AWS S3. Provides cloud-based file upload and storage.

Real-time: Socket.io / WebSockets. Supports real-time updates and notifications.

API Documentation: Swagger / OpenAPI. Provides API documentation and testing support.

Deployment: Vercel / AWS. Provides cloud hosting and deployment.

4.2 API Design

API Versioning

/api/v1/[resource]

Standard API Response Format

Successful responses use a consistent structure containing a success indicator, a descriptive message, returned data, and metadata such as the timestamp and API version. Error responses similarly contain a success indicator, an error message, an error code and details, together with metadata.

4.3 Database Design

Core Tables

users: User accounts and authentication (id, name, email, password_hash, role, avatar).

projects: Project management (id, name, description, progress, status, start_date, end_date).

tasks: Task management (id, title, description, status, priority, due_date, assigned_to).

team_members: Project team assignments (id, user_id, project_id, role, joined_at).

comments: Task discussions (id, task_id, user_id, content, parent_id, created_at).

attachments: File attachments (id, task_id, file_name, file_url, file_size, uploaded_by).

notifications: User notifications (id, user_id, type, message, read, link, created_at).

team_activity: Activity logs (id, project_id, user_id, action, details, created_at).

Data Relationships

The database uses one-to-many relationships such as Project to Tasks, User to Comments, and Task to Attachments. It also supports a many-to-many relationship between Users and Projects through the team_members table, as well as a one-to-one relationship between a User and Profile.

4.4 Security

Authentication

Authentication uses JSON Web Tokens (JWT) to provide stateless authentication. Tokens will have an expiration mechanism and a refresh process, while passwords will be securely hashed using bcrypt.

Authorization

Authorization is handled through role-based access control, permission-based access control, and resource-level permissions. This ensures that users can only perform actions and access resources permitted by their role and assigned responsibilities.

Data Protection

Data protection measures include input validation and sanitization, SQL injection prevention, XSS protection, appropriate CORS configuration, rate limiting, and HTTPS enforcement.

4.5 Performance

Caching Strategy

Redis will be used for session storage and frequently accessed data. API response caching and database query caching will also be used where appropriate to reduce repeated processing and improve response times.

Database Optimization

Database performance will be improved through indexing of frequently queried fields, query optimization, connection pooling, and the use of read replicas where scaling requires them.

5. Development Phases

Foundation. Project setup and configuration, database schema design and migration, basic authentication, and the core API structure.

Core Features. User management endpoints, task CRUD operations, project management endpoints, and team member management.

Collaboration Features. Comments and discussions, file upload and management, notifications, and the activity feed.

Advanced Features. Real-time updates, analytics and reporting, search functionality, task dependencies, and subtasks.

Optimization & Testing. Performance optimization, security hardening, load testing, and API documentation.

Deployment. Production deployment, monitoring setup, CI/CD pipeline, and backup and disaster recovery.

6. Deliverables

6.1 Documentation

The documentation deliverables include a complete Swagger/OpenAPI specification, database schema documentation with ER diagrams and table descriptions, architecture documentation containing system design and component diagrams, a developer guide covering setup instructions and coding standards, and user manuals for administrators and users.

6.2 Codebase

The codebase deliverables include the complete TypeScript backend source code, database migration scripts for all environments, and seed data for testing and demonstration purposes.

6.3 Testing

Testing deliverables include a comprehensive unit test suite, integration tests for API endpoints, load tests for performance and scalability, and security tests for vulnerability scanning.

7. Success Metrics

7.1 Technical Metrics

Technical success will be measured using an API response time of less than 200 milliseconds for 95% of requests, uptime above 99.9%, support for more than 1,000 concurrent users, 100% ACID compliance for data consistency, and zero critical security vulnerabilities.

7.2 Business Metrics

Business success will be measured through user adoption of more than 500 active users within the first three months, more than 1,000 tasks created per week, at least 50 active teams, and a user satisfaction rate above 85%.

8. Risks and Mitigation

Performance bottlenecks (High impact): Implement caching, database optimization, and load testing.

Security vulnerabilities (Critical impact): Conduct regular security audits and penetration testing while maintaining strong input validation.

Data loss (High impact): Maintain regular backups, a disaster recovery plan, and replication.

Scaling issues (Medium impact): Prepare a horizontal scaling plan and consider a microservices architecture as growth requires.

Team availability (Medium impact): Maintain clear documentation, knowledge transfer, and backup team members.

9. Maintenance and Support

9.1 Ongoing Maintenance

Ongoing maintenance will include regular security updates, bug fixes and patches, performance monitoring, database maintenance, and log analysis. These activities will help keep the backend secure, stable, and responsive as the platform evolves.

9.2 Support Plan

Support will be organized into three levels. Tier 1 will provide documentation and a knowledge base for common issues. Tier 2 will provide support team responses within 24 hours. Tier 3 will address critical issues with a response target of within four hours.

10. Timeline

Planning & Design: 2 weeks

Foundation: 2 weeks

Core Features: 3 weeks

Collaboration: 2 weeks

Advanced: 3 weeks

Testing: 2 weeks

Deployment: 2 weeks

Total: 16 weeks (~4 months)